

제5강

1) useReducer를 이용한 입출금

- 각 버튼을 클릭할 때 마다 금액만큼 입금, 출금하고 0로 리셋 하고, 이율만큼 금액을 가산하는 프로그램을 작성한다.

[src]-[components05] BankPage.jsx

```
import React, { useState, useReducer, useRef } from 'react'
import './Style05.css'

const ACTION_TYPE = {
  deposit: '입금',
  withdrawal: '출금',
  interest: '이자',
  close: '해지'
}

const initState = 10000; //초기 잔액
const rate = 0.1; //이율

const reducer = (state, action) => {
  const amount = parseInt(action.amount);

  switch(action.type){
    case ACTION_TYPE.deposit:
      return state + amount;
    case ACTION_TYPE.withdrawal:
      if(state < amount){
        alert('잔액이 부족합니다!');
        return state;
      }
      return state - amount;
    case ACTION_TYPE.interest:
      return state + (state * rate);
    case ACTION_TYPE.close:
      return 0;
    default:
      return state;
  }
}

const BankPage = () => {
  const [amount, setAmount] = useState(1000);
  const [state, dispatch] = useReducer(reducer, initState);
  return (
    <div className='box'>
      <h1>잔액:{parseInt(state).toLocaleString()}원</h1>
      <div>
        <span>금액:</span>
        <input value={money}
          onChange={(e)=>setAmount(e.target.value)}
          placeholder='금액' type='number' step={1000}/>
      </div>
      <div>
        <button onClick={()=>dispatch( {type:ACTION_TYPE.deposit, amount} )}>입금</button>
        <button onClick={()=>dispatch( {type:ACTION_TYPE.withdrawal, amount} )}>출금</button>
        <button onClick={()=>dispatch( {type:ACTION_TYPE.interest} )}>이자</button>
        <button onClick={()=>dispatch( {type:ACTION_TYPE.close} )}>해지</button>
      </div>
    </div>
  )
}

export default BankPage
```

2) useReducer를 이용한 장바구니

1. 등록 : 장바구니 상품수를 1증가하고 장바구니 목록에 상품을 등록한다.
2. 삭제 : 장바구니 상품수를 1감소하고 장바구니 목록에서 해당 상품을 삭제한다.
3. 주문 : 장바구니 상품수를 1감소하고 장바구니 목록에서 상품을 삭제하고 주문 상품수를 1증가하고 주문 목록에 상품을 등록한다.
4. 취소 : 주문 상품수를 1감소하고 주문 목록에서 상품을 삭제하고 장바구니 상품수를 1증가하고 장바구니 목록에 상품을 등록한다.

[src]-[components05] CartPage.jsx

```
import React, { useReducer, useState, useRef } from 'react'
import Product from './Product'
import './Style05.css'

export const ACTION_TYPE = {
  add:'등록',
  delete:'삭제',
  order:'주문',
  cancel:'취소'
}

const initState = {
  cart: {
    count:1,
    products:[{ id:3, name:'삼성 냉장고'}]
  },
  order: {
    count:2,
    products:[{ id:1, name:'엘지 냉장고'}, {id:2, name:'엘지 세탁기'}]
  }
}

const reducer = (state, action) => {
}

const CartPage = () => {
  const [name, setName] = useState('삼성 세탁기');
  const [state, dispatch] = useReducer(reducer, initState);
  const idRef = useRef(4);

  return (
    <div className='box'>
      <div>
        <h1>카트 목록</h1>
        <h5>상품수:{state.cart.count}개</h5>
        <input value={name} onChange={(e)=>setName(e.target.value)} placeholder='상품명' />
        <button>등록</button>
        <table>
          <tbody>
            { state.cart.products.map(p=>
              <Product key={p.id} product={p} type='cart' dispatch={dispatch}/>
            )}
          </tbody>
        </table>
      </div>
      <div>
        <h1>주문 목록</h1>
        <h5>상품수:{state.order.count}개</h5>
        <table>
          <tbody>
            { state.order.products.map(p=>
              <Product key={p.id} product={p} type='order' dispatch={dispatch}/>
            )}
          </tbody>
        </table>
      </div>
    </div>
  )
}

export default CartPage
```

[src]-[components05] CartPage.jsx

```
const reducer = (state, action) => {
  switch(action.type){
    case ACTION_TYPE.add:
      return {
        ...state,
        cart:{
          count:state.cart.count + 1,
          products:state.cart.products.concat(action.product)
        }
      }
    case ACTION_TYPE.delete:
      return {
        ...state,
        cart:{
          count:state.cart.count - 1,
          products:state.cart.products.filter(p=>p.id !== action.product.id)
        }
      }
    case ACTION_TYPE.order:
      return {
        ...state,
        cart:{
          count:state.cart.count - 1,
          products:state.cart.products.filter(p=>p.id !== action.product.id)
        },
        order:{
          count:state.order.count + 1,
          products:state.order.products.concat(action.product)
        }
      }
    case ACTION_TYPE.cancel:
      return {
        ...state,
        cart:{
          count:state.cart.count + 1,
          products:state.cart.products.concat(action.product)
        },
        order:{
          count:state.order.count - 1,
          products:state.order.products.filter(p=>p.id !== action.product.id)
        }
      }
  }
}

<button onClick={()=>dispatch({type:ACTION_TYPE.add, product:{id:idRef.current++, name}})}>등록</button>
```

[src]-[components05] Product.jsx

```
import { ACTION_TYPE } from './CartPage';
const Product = ({product, type, dispatch}) => {
  const { id, name } = product;
  return (
    <tr>
      <td width='50px'>{id}</td>
      <td width='270px'>{name}</td>
      { type=='cart' &&
        <td>
          <button onClick={()=>dispatch({type:ACTION_TYPE.delete, product})}>삭제</button>
          <button onClick={()=>dispatch({type:ACTION_TYPE.order, product})}>주문</button>
        </td> }
      { type=='order' &&
        <td>
          <button onClick={()=>dispatch({type:ACTION_TYPE.cancel, product})}>취소</button>
        </td> }
    </tr>
  )
}
export default Product
```